



Indian Institute of Technology Guwahati

BSc. (Hons.) Data Science & Artificial Intelligence
Online Degree Programme

INTERNSHIP/TERM PROJECT

ENGINEERING INTELLIGENCE: A MULTIMODAL AI FRAMEWORK FOR ENGINEERING, TECHNICAL, SCIENTIFIC, MATHEMATICAL ANALYSIS AND INTELLIGENT ASSISTANCE

Student Name:	Sagar Sant
Student ID :	23035010233
Email ID :	sagar.sant@op.iitg.ac.in
Programme :	BSc. (Hons.) Data Science & AI
Course Code :	DA 379
Course Name :	Internship-III
Trimester :	IX

Problem Statement

0:30–1:30

Why technical AI assistance needs more than a single chatbot

Heterogeneous inputs

Natural-language questions, technical documents, equations, numerical data, drawings, schematics and figures.

Fragmented tooling

Different tasks are often handled by different applications: document search, calculation, visual analysis and general AI.

Reliability challenge

Language models are useful for reasoning, but exact numerical computation and source-specific technical knowledge benefit from dedicated tools.

THE GAP

- A unified system should understand the request before deciding how to process it.
- The framework must combine reasoning with retrieval, deterministic computation and multimodal analysis.
- The same interaction layer should support engineering, technical, scientific and mathematical assistance.

Project goal: route the right problem to the right model, retrieval path or computational tool.

Project Objectives

1:00–1:30

From unified assistant to verifiable technical workflows

Unified AI

Design one framework for engineering, technical, scientific and mathematical assistance.

NLP + LLMs

Use multiple local language models with task-aware selection.

Document intelligence

Process, embed, retrieve and ground answers in technical documents.

Reliable computation

Use deterministic numerical and symbolic tools where exact computation is required.

Multimodal analysis

Interpret engineering images, drawings, schematics and technical figures.

Orchestration

Coordinate specialized paths through LangGraph and supporting LangChain components.

Technology & Development Stack

1:30–2:15

The complete implementation is Python-based and developed in Visual Studio Code

Development

Visual Studio Code
Python

AI Application Layer

LangChain
LangGraph

Local Model Serving

Ollama
Multiple local LLM / vision models

Retrieval

Embeddings • Vector store
RAG / document pipeline

Engineering Computation

NumPy • SciPy • SymPy
Deterministic calculation tools

Interface

Streamlit
Structured technical output

LOCAL MODEL ECOSYSTEM

Examples used across the project: Gemma • Qwen • DeepSeek • Llama • Mistral • Phi • Qwen-VL

Test Inputs & Evaluation Scope

1:30–2:15

The project was evaluated as an integrated application rather than a conventional supervised ML training task

Natural-language technical questions

Engineering, science, mathematics and general technical queries.

Technical documents

PDFs and document-based questions requiring extraction and retrieval.

Engineering images

Drawings, schematics, diagrams, figures and other visual technical content.

Mathematical / scientific expressions

Arithmetic, formulas, symbolic expressions and engineering calculations.

Evaluation focus

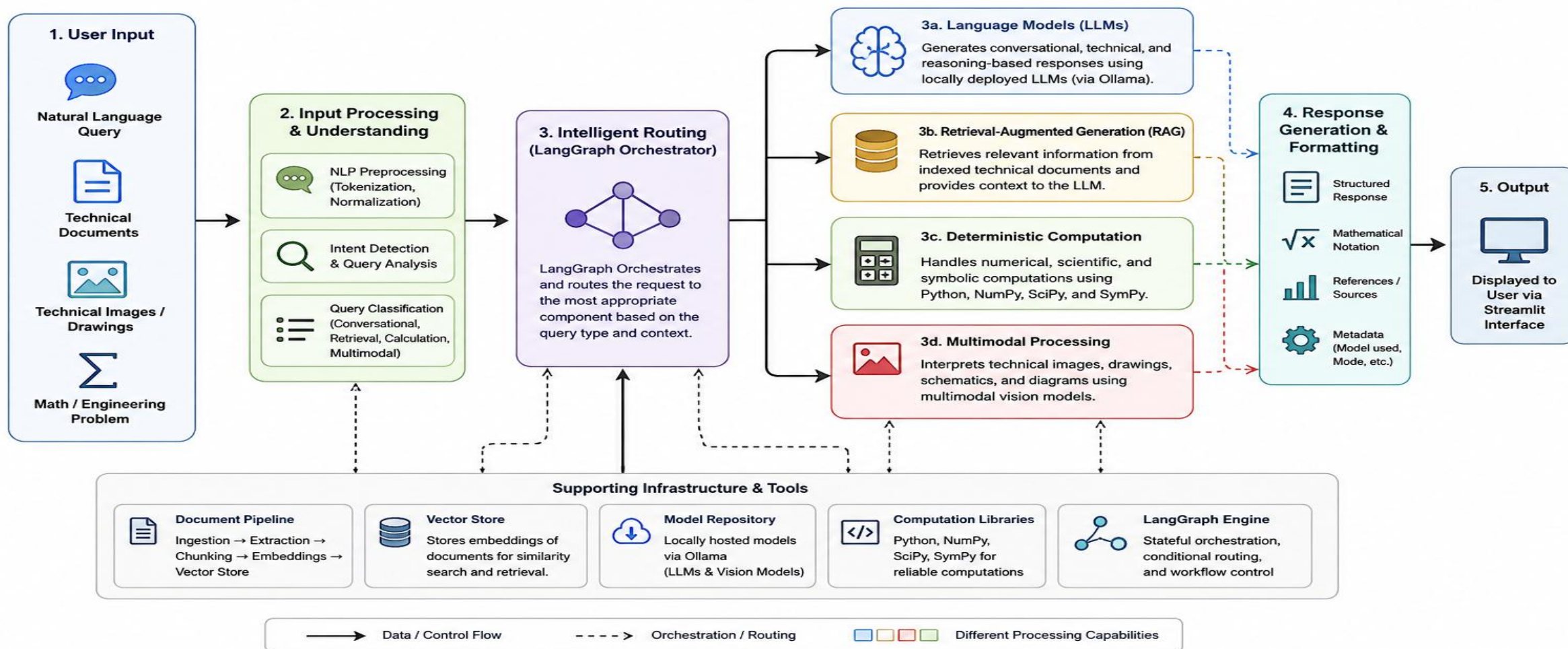
Functional correctness • routing • deterministic computation • document integration • multimodal execution • end-to-end application behavior

Overall Workflow & Architecture

2:15-3:00

One request, multiple specialized execution paths

Engineering Intelligence: Overall Workflow and Architecture



Code Walkthrough — Core Project Implementation

3:00–4:15

Key Python modules and the role of each component

Core project structure

```
src/  
├── app/  
│   └── document_service.py  
├── llm/  
│   ├── model_router.py  
│   └── ollama/  
│       └── ollama_llm.py  
├── orchestration/  
│   └── engineering_graph.py  
└── tools/  
    ├── calculator.py  
    └── calculation_planner.py
```

DocumentService

Central application service: conversation, documents, retrieval, calculation and response generation.

ModelRouter

Selects the appropriate model for the request type and complexity.

EngineeringGraph

LangGraph orchestration layer with conditional routing.

Calculator

Deterministic numerical and symbolic computation.

CalculationPlanner

Structures complex engineering calculations before execution.

Ollama LLM Layer

Local model invocation for text and multimodal generation.

Python modules → LangChain components → LangGraph workflow → Streamlit application

Code Walkthrough — Deterministic Calculation Path

4:00–4:45

Language reasoning identifies the task; deterministic tools perform the computation

Calculation routing logic

```
if not self._is_calculation_question(question):
    return None

fast = calculate(question)

if fast is not None:
    return {
        "plan": {"type": "expression"},
        "calculation": fast,
        "fast_path": True,
    }

solved = self.calculation_planner.solve(question)
solved["fast_path"] = False
```

Fast path

Simple supported expressions go directly to the deterministic calculator.

Planner path

More complex problems are structured by the calculation planner before deterministic execution.

Verified example

$7425^3 \rightarrow 409,344,890,625$

REASONING → ROUTING → DETERMINISTIC EXECUTION → EXPLANATION

Retrieval-Augmented & Multimodal Processing

4:45–5:30

Two major extensions beyond ordinary text-only chat

Document → RAG

Upload document → extract content → chunk → embeddings
→ vector retrieval → supply relevant context → generate
grounded response.

Image → Vision

Upload technical image → visual interpretation → combine
visual information with natural-language instructions →
generate technical response.

Grounding

Use retrieved document evidence when
the answer depends on source material.

Context

Preserve technical terminology, labels,
equations and document-specific
information.

Multimodality

Extend assistance to drawings,
schematics, diagrams and figures.

Engineering Intelligence — Application

5:30–6:15

A unified Streamlit interface for technical interaction

Deploy

Engineering Intelligence

Upload

Upload engineering PDFs or images. Analyze text, tables, drawings, schematics, labels and figures.

Upload engineering documents or images



Upload

200MB per file • PDF, PNG, JPG, WEBP

No files selected.

Analyze Files

Clear Chat

Clear Indexed Documents

LOCAL-FIRST • MULTIMODAL • ENGINEERING AI

Engineering Intelligence

Your AI assistant for engineering, science and mathematics — analyze manuals, drawings, schematics, figures, formulas and technical data, then ask questions in natural language.

Documents & RAG

Vision & Schematics

Σ Engineering Calculations

Current Information



Engineering Documents

Search manuals, specifications, tables, procedures and technical notes with document-aware retrieval.



Drawings & Schematics

Analyze components, labels, connections, figures, diagrams and other engineering visuals.



Math & Engineering

Work through formulas, equations, units, calculations and technical explanations with formatted notation.



Current Information

Use web research when a question depends on current, recent or changing information.

START A CONVERSATION

Upload a file when your question depends on a document or image — or simply ask an engineering, science or mathematics question.

Try:

- Explain the schematic and trace the conventional current path.

Ask an engineering, science, math or document question...



Results & Outputs

6:15–7:15

Functional evaluation across the integrated system

Capability	Evaluation	Result
Conversational AI	General conversational queries	Successful
Technical QA	Engineering and scientific queries	Successful
Mathematical computation	7425 ³	Exact result
Calculation routing	Quantitative query detection	Correct route
Calculation planning	Structured computational problem	Successful
Document intelligence	Document-service integration	Successful
Model routing	Multiple query categories	Successful
LangGraph orchestration	Calculation workflow execution	Successful
Multimodal processing	Technical visual inputs	Successful
Streamlit interface	End-to-end application testing	Successful

10 / 10 LISTED FUNCTIONAL CHECKS PASSED

Evaluation & Verification

7:15–8:00

Objective checks used to validate the integrated application

Routing correctness — Verified

Request categories reached the intended processing paths.

Deterministic computation — Verified

Exact result confirmed for representative calculations.

Document integration — Verified

Document service and retrieval path executed successfully.

Multimodal processing — Verified

Technical visual inputs could be processed end-to-end.

Application integration — Verified

Streamlit interface connected the major capabilities.

Discussion — Strengths & Trade-offs

8:00–8:45

Why the architecture is useful and where it still has limits

Strength: task specialization

Different requests can use different models, tools and retrieval paths instead of one model doing everything.

Strength: computational reliability

Deterministic computation reduces dependence on language-model arithmetic for supported mathematical tasks.

Strength: source grounding

RAG provides a route for answers to use retrieved technical document context.

Trade-off: complexity

Multiple models, retrieval and orchestration increase system complexity and resource requirements.

Current limitation

The present evaluation is primarily functional; broader benchmark datasets, controlled latency testing and quantitative task metrics are future work.

Conclusion & Future Work

8:45–9:45

Final takeaways for the project

What was achieved

- Unified engineering, technical, scientific and mathematical assistance
- Multiple local LLMs with intelligent routing
- RAG and technical document intelligence
- Deterministic numerical and symbolic computation
- Multimodal engineering-image analysis
- LangChain components + LangGraph orchestration
- Interactive Streamlit application

Future work

- Larger and more diverse evaluation sets
- Quantitative benchmarks and latency measurement
- More engineering-specific calculation and simulation tools
- Stronger multimodal reasoning for complex drawings
- Expanded coding / engineering tool integrations
- Production-scale deployment and monitoring

Key takeaway: Engineering assistance becomes more capable when LLM reasoning is connected to the right retrieval, computation and multimodal tools.

Wrap Up

9:45–10:00

Engineering Intelligence in one view

UNDERSTAND

NLP + LLMs

RETRIEVE

RAG + Documents

COMPUTE

Python + NumPy + SciPy + SymPy

SEE

Multimodal Vision

One application. Multiple specialist capabilities. One unified engineering intelligence layer.

LangChain • LangGraph • Ollama • RAG • SymPy • NumPy • SciPy • Vision • Streamlit

Thank You

Questions & Demonstration